

Ultimate Spring Core Mastery & Interview Execution Blueprint

Complete Curriculum Notes, Hands-On Debugging Labs, Real-World Architecture & Interview Cracking Strategy

Enterprise Preparation Edition | All 11 Core Modules + Deep-Dive Edge Cases + Hands-On Practice Projects

SECTION I: SPRING CORE FOUNDATIONS

Module 1: Spring Fundamentals & Architecture

1. What is Spring Framework? (Monolith vs. Modular Architecture)

The **Spring Framework** is an open-source, lightweight Java enterprise framework designed to simplify enterprise application development through Inversion of Control (IoC) and Dependency Injection (DI). Unlike traditional legacy Java EE monoliths that forced heavy dependencies on full application servers (like WebSphere or WebLogic), Spring adopts a **modular architecture**. You only import the modules you need (e.g., `spring-core`, `spring-context`, `spring-web`, `spring-tx`).

2. Tight Coupling vs. Loose Coupling Mechanics

- **Tight Coupling:** Occurs when a class instantiates its dependencies directly using the `new` keyword. Changing or mocking the dependency requires modifying the calling class.
- **Loose Coupling:** Achieved by programming to **interfaces** and allowing an external container (Spring IoC) to inject the appropriate concrete implementation at runtime.

REAL-WORLD ENTERPRISE SCENARIO: PAYMENT PROCESSING SYSTEM

In a financial platform, an `OrderService` needs to charge customer credit cards. Hardcoding `StripePaymentService` directly inside `OrderService` makes it impossible to switch to `PayPal` or supply a mock implementation during automated integration testing.

```
// --- TIGHTLY COUPLED (BAD PRACTICE) ---
class OrderServiceBad {
    private StripeProcessor processor = new StripeProcessor();
    public void checkout(double amount) { processor.pay(amount); }
}

// --- LOOSELY COUPLED (SPRING CORE BEST PRACTICE) ---
public interface PaymentProcessor {
    void processPayment(double amount);
}

@Component("stripeProcessor")
public class StripeProcessor implements PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing $" + amount + " via Stripe.");
    }
}

@Component("paypalProcessor")
public class PaypalProcessor implements PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing $" + amount + " via PayPal.");
    }
}

@Service
public class OrderService {
    private final PaymentProcessor paymentProcessor;

    public OrderService(@Qualifier("stripeProcessor") PaymentProcessor paymentProcessor) {
        this.paymentProcessor = paymentProcessor;
    }

    public void checkout(double amount) {
        paymentProcessor.processPayment(amount);
    }
}
```

3. POJO, POJI, and JavaBean Concepts

- **POJO (Plain Old Java Object):** Any standard Java object not constrained by framework-specific class inheritance.
- **POJI (Plain Old Java Interface):** A standard Java interface free of framework imports.
- **JavaBean:** A POJO adhering to strict conventions: public no-arg constructor, private encapsulated attributes, getters/setters, and implementing `java.io.Serializable`.

4. Spring Framework Core Modules Overview

- `spring-core` : Basic utilities, resource loaders, and core framework primitives.
- `spring-beans` : Contains `BeanFactory`, handling bean definition parsing, manipulation, and lifecycle creation.
- `spring-context` : Extends `spring-beans` with `ApplicationContext`, adding internationalization (i18n), event publishing, and environment profiles.
- `spring-aop` : Provides proxy-based Aspect-Oriented Programming infrastructure.

MODULE 1 MOCK INTERVIEW & ENTERPRISE Q&A

Q: Why is Spring considered lightweight compared to traditional Java EE?

A: Spring allows developers to write code using simple POJOs without inheriting heavy framework classes. It can run on standard Servlet containers (like Tomcat) rather than requiring full Java EE application servers.

Module 2: Inversion of Control (IoC) & Containers

1. What is Inversion of Control (IoC)?

Inversion of Control is a design principle where control over object creation, configuration, and lifecycle management is transferred from application code to a centralized container framework.

2. Dependency Lookup vs. Dependency Injection

- **Dependency Lookup (DL):** The component proactively queries a container registry to locate its dependency (e.g., `context.getBean("myBean")`). *Disadvantage:* Couples code to container APIs.
- **Dependency Injection (DI):** The container automatically injects dependencies into the component during instantiation via constructors, setters, or fields.

3. IoC Container Architecture: BeanFactory vs. ApplicationContext

Feature	BeanFactory	ApplicationContext
Instantiation Strategy	Lazy (Beans created on demand via <code>getBean()</code>).	Eager (Singleton beans created during startup).
Enterprise Features	Basic bean instantiation and DI support.	Full support for AOP, <code>ApplicationEvent</code> , i18n, and web contexts.
Target Resource Level	Lightweight / Resource-constrained environments.	Enterprise web apps & microservices (Default standard).

```
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class ContainerDemo {
    public static void main(String[] args) {
        var annotationContext = new AnnotationConfigApplicationContext(AppConfig.class);
        System.out.println("IoC Container successfully started.");
        annotationContext.close();
    }
}
```

Module 3: Dependency Injection & Configuration Mechanics

1. Setter Injection vs. Constructor Injection vs. Field Injection

- **Constructor Injection (Recommended):** Enforces immutability via `final` fields, prevents `NullPointerException`s, and makes isolated unit testing straightforward without loading a Spring Context.
- **Setter Injection:** Ideal for optional dependencies that can be reconfigured dynamically at runtime.
- **Field Injection (Discouraged):** Uses `@Autowired` on private fields directly. *Drawbacks:* Hides dependencies, prevents immutability, requires reflection for testing.

```
@Configuration
@PropertySource("classpath:application.properties")
public class AppConfig {

    @Value("${app.timeout:5000}")
    private int timeout;

    @Value("#{T(java.lang.Math).random() * 100}")
    private double randomNumber;

    @Bean
    public DatabaseConnection dbConnection() {
        return new DatabaseConnection(timeout, randomNumber);
    }
}
```

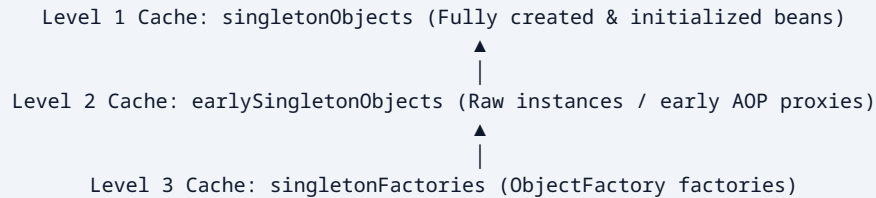
Module 4: Autowiring & Ambiguity Resolution

1. Annotation Autowiring & Ambiguity Handling

When Spring finds multiple beans of the same type matching an injection point, it throws `NoUniqueBeanDefinitionException`. Resolution strategies include `@Primary` and `@Qualifier("beanName")`.

2. Circular Dependency Mechanics & The 3-Level Cache

Spring resolves circular dependencies for singleton field/setter injections using a three-level cache in `DefaultSingletonBeanRegistry`:



MODULE 4 MOCK INTERVIEW & ENTERPRISE Q&A

Q: Can Spring's 3-level cache solve constructor-based circular dependencies?

A: No! Constructor-based dependencies require arguments during object instantiation, before any factory can be registered in the Level 3 cache. Resolution requires using `@Lazy` on one constructor argument or refactoring the dependency hierarchy.

Module 5: Bean Scopes & Lifecycle Mechanics

1. Core & Web-Aware Scopes

- `singleton` (Default): One single shared instance per IoC container context.
- `prototype`: A new instance is created every time the bean is requested.
- `request`, `session`, `websocket`: Web-aware scopes for HTTP lifecycles.

2. Prototype Bean Injected into Singleton Host (Scope Proxy / @Lookup)

```
@Component
@Scope(value = "prototype", proxyMode = ScopedProxyMode.TARGET_CLASS)
class Token {
    private final String id = UUID.randomUUID().toString();
    public String getId() { return id; }
}

@Component
class TokenService {
    private final Token token;
    public TokenService(Token token) { this.token = token; }

    @Lookup
    public Token getFreshToken() { return null; } // Dynamic CGLIB override at runtime
}
```

Module 6 & 7: Component Scanning, Profiles & SpEL

```
// --- CUSTOM CONDITION & EVENT HANDLING ---
public record PaymentEvent(String transactionId, double amount) {}

@Service
public class PaymentPublisher {
    private final ApplicationEventPublisher publisher;

    public PaymentPublisher(ApplicationEventPublisher publisher) {
        this.publisher = publisher;
    }

    public void processPayment(String id, double amount) {
        publisher.publishEvent(new PaymentEvent(id, amount));
    }
}

@Component
public class AuditListener {
    @EventListener
    public void onPayment(PaymentEvent event) {
        System.out.println("[AUDIT EVENT] Payment processed: " + event.transactionId());
    }
}
```

SECTION II: ADVANCED & ENTERPRISE MODULES

Module 8 - 11: Data JDBC, Transactions, AOP & Spring Boot

```
// --- MODULE 9: TRANSACTIONS & MODULE 10: AOP ---
@Service
public class AccountService {

    @Transactional(
        propagation = Propagation.REQUIRED,
        isolation = Isolation.READ_COMMITTED,
        rollbackFor = Exception.class
    )
    public void transferMoney(Long source, Long target, double amount) {
        // Business logic execution inside database transaction
    }
}

@Aspect
@Component
public class LoggingAspect {

    @Around("execution(* com.example.service..*(..))")
    public Object measureExecutionTime(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.currentTimeMillis();
        Object output = pjp.proceed();
        long duration = System.currentTimeMillis() - start;
        System.out.println("Execution time: " + duration + " ms");
        return output;
    }
}
```

MODULE 9 INTERVIEW PITFALL ALERT

Q: What happens if a method inside a class calls a `@Transactional` method in the SAME class?

A: Self-Invocation Bug! The inner call bypasses the Spring AOP proxy, executing directly as a plain method on `this`, causing `@Transactional` annotations to be completely ignored.

SECTION III: HANDS-ON DEBUGGING LABS & PRACTICAL PROJECTS

1. Practical Debugging Labs for Common Spring Failures

LAB 1: REPRODUCING & FIXING THE SELF-INVOCATION PROXY BUG

The Problem: In an e-commerce platform, calling `updateStock()` annotated with `@Transactional` from inside `processOrder()` within the same service class fails to create a database transaction, leaving stock uncommitted on errors.

```
// --- BUGGY IMPLEMENTATION (SELF-INVOCATION) ---
@Service
public class InventoryServiceBug {
    public void processOrder() {
        // Calling internal method directly on 'this' bypasses Spring's AOP Proxy!
        this.updateStock();
    }

    @Transactional
    public void updateStock() {
        // Transaction logic here fails to execute in a transactional context
    }
}

// --- FIXED IMPLEMENTATION (DESIRED ARCHITECTURE) ---
@Service
public class InventoryServiceFixed {
    private final StockRepository stockRepository;

    public InventoryServiceFixed(StockRepository stockRepository) {
        this.stockRepository = stockRepository;
    }

    @Transactional // Moved transaction boundary to public entry point or separate bean
    public void processOrderAndStock() {
        stockRepository.updateStock();
    }
}
```

LAB 2: RESOLVING PROTOTYPE SCOPE CREEP IN SINGLETON SERVICES

The Problem: Injecting a prototype bean into a singleton service directly results in the prototype bean being instantiated only once, losing its prototype behavior.

```
// --- SOLUTION: USING @LOOKUP METHOD INJECTION ---
@Component
@Scope("prototype")
class TaskRunner {
    private final String taskId = UUID.randomUUID().toString();
    public void run() { System.out.println("Running Task ID: " + taskId); }
}

@Service
abstract class TaskSchedulerService {

    // Spring overrides this abstract method at runtime via CGLIB subclassing
    @Lookup
    public abstract TaskRunner getTaskRunner();

    public void executeSchedule() {
        TaskRunner runner = getTaskRunner(); // Retrieves a FRESH prototype instance!
        runner.run();
    }
}
```

SECTION IV: COMPLETE INTERVIEW EXECUTION BLUEPRINT

1. The 4-Step STAR Method Framework for Technical Scenarios

HOW TO STRUCTURE SCENARIO RESPONSES

When asked scenario-based questions (e.g., *"How did you resolve performance bottlenecks or bean exceptions in your project?"*), structure your response using **STAR**:

- **S - Situation:** Set the context ("In our microservice API, we hit a `BeanCurrentlyInCreationException` during startup.>").
- **T - Task:** Explain your objective ("I needed to resolve the circular dependency without compromising constructor immutability.>").
- **A - Action:** Describe your precise technical solution ("I refactored the design to extract common business logic into a dedicated helper service, eliminating the circular loop.>").
- **R - Result:** Share measurable outcomes ("Startup time dropped by 20%, and all integration tests passed without `@Lazy` hacks.>").

2. Master Interview Question & Answer Matrix

Topic Area	Interview Question	Winning Answer Breakdown
Circular Dependency	How does Spring solve circular dependencies using the 3-level cache?	Spring exposes an <code>ObjectFactory</code> in the Level 3 cache (<code>singletonFactories</code>) during bean creation. When a circular dependency occurs, the second bean fetches the early reference from Level 3, promotes it to Level 2 (<code>earlySingletonObjects</code>), and completes initialization cleanly.
Prototype in Singleton	What happens when you inject a prototype bean into a singleton, and how do you fix it?	The prototype is injected only once during singleton initialization. Fix it using Scoped Proxies (<code>proxyMode = ScopedProxyMode.TARGET_CLASS</code>) or <code>@Lookup</code> method injection.
Bean Lifecycle & AOP	What is the role of BeanPostProcessor in creating Spring AOP proxies?	During the <code>postProcessAfterInitialization()</code> lifecycle phase, AOP auto-proxy creators inspect beans against pointcut definitions and replace target instances with dynamic proxies (JDK or CGLIB).
Transactions	How does <code>@Transactional</code> work internally using Spring AOP proxies?	The proxy intercepts method calls, opens a DB transaction via <code>PlatformTransactionManager</code> , executes target logic, and commits or rolls back based on unchecked exceptions.
Auto-Configuration	How does <code>@EnableAutoConfiguration</code> read META-INF configurations?	It uses <code>AutoConfigurationImportSelector</code> to read class names from <code>META-INF/spring.factories</code> or <code>org.springframework.boot.autoconfigure.AutoConfiguration.imports</code> , evaluating <code>@ConditionalOnClass</code> rules.

FINAL INTERVIEW ROADMAP STRATEGY

- 1. Revisit the PDF Regularly:** Use this document as your primary cheat sheet before technical rounds.
- 2. Write Annotations Without IDE Auto-Completion:** Practice writing `@RestController`, `@RequestMapping`, and `@Transactional` signatures by hand.
- 3. Explain Mechanics Out Loud:** Articulate under-the-hood terms like `BeanPostProcessor` and `CGLIB Subclassing` during mock practice.